

Security Assessment Report

Generated: 2026-05-05 10:54

Report ID: 8110cf13

Security Assessment Report

Target: /workspace/uploads/704086176943

Assessment Basis: Automated static analysis results (Bandit, SonarQube, Safety)

Overall Risk Rating: Low

Scope Note: Findings are limited to the automated scan results provided. No confirmed application vulnerabilities were identified in the scanned Python source, and no dependency inventory was available for package vulnerability assessment.

1. Executive Summary

The automated security review of the provided Python project did not identify any confirmed vulnerabilities in the scanned source file (server.py) via Bandit or SonarQube. Bandit reported zero issues across 447 lines of code, and SonarQube returned no issues.

The dependency security scan could not be completed because no requirements.txt file was present in the project directory. As a result, thirdparty package risk could not be evaluated, leaving a potential blind spot in the overall security posture.

Based on the available evidence, the application currently appears to have a low observed risk level from a sourcecode perspective. However, the absence of dependency metadata reduces assessment completeness and should be addressed to ensure supplychain risks are not overlooked.

2. Risk Overview

Findings by Severity

Critical: 0

High: 0

Medium: 0

Low: 0

Informational: 1

Risk Interpretation

Critical / High / Medium / Low: No confirmed codelevel vulnerabilities were detected in the automated scan output.

Informational: One assessment limitation was identified: dependency vulnerability analysis could not be performed because no requirements.txt file was found.

This means the current scan results do not indicate an immediate exploitable weakness in the analyzed source, but the security assessment is incomplete until dependencies are documented and reviewed.

3. Detailed Findings

Informational Findings

Finding 1 — Missing Dependency Manifest Prevented Package Vulnerability Analysis

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A06: Vulnerable and Outdated Components

Affected Component:

/workspace/uploads/704086176943 (projectlevel dependency inventory)

Absence of requirements.txt

Description:

The project directory does not contain a requirements.txt file, which prevented the Safety tool from performing dependency vulnerability analysis.

This is not a direct software vulnerability in the application logic, but it is a security assessment gap because installed thirdparty packages could contain known vulnerabilities, outdated releases, or insecure transitive dependencies.

Dependency visibility is essential for identifying supplychain exposure and maintaining secure software composition.

Impact:

An attacker may be able to exploit a vulnerable thirdparty library if one is present in the

runtime environment.

Business impact includes increased exposure to known CVEs, reduced confidence in the software supply chain, and delayed remediation of externally introduced risks.

The absence of a manifest also complicates patch management and reproducible builds.

Proof of Concept (if possible):

Not applicable. This is an assessment limitation rather than an exploitable weakness.

If the project uses untracked dependencies, a vulnerability scanner cannot validate whether installed packages are affected by public CVEs.

Recommended Remediation:

Add and maintain a dependency manifest such as requirements.txt, pyproject.toml, or Pipfile.

Pin versions of all direct dependencies and review transitive dependencies regularly.

Integrate dependency scanning into CI/CD using tools such as Safety, pipaudit, or Dependabot.

Establish a patching process for updating libraries with known vulnerabilities.

Use lockfiles where appropriate to ensure reproducible, auditable builds.

4. Recommendations (Global)

Maintain a complete dependency inventory: Ensure all thirdparty packages are documented and versionpinned.

Automate dependency security checks: Integrate package vulnerability scanning into the build pipeline.

Continue secure coding practices: The current static analysis results are clean, but code should continue to follow securebydefault principles.

Validate inputs and outputs: Preserve strong input validation, especially for any web, API, or fileprocessing functionality.

Review authentication and authorization controls: Confirm access control is enforced consistently across all routes and actions.

Use leastprivilege principles: Limit service account permissions, filesystem access, and runtime privileges.

Keep frameworks and libraries current: Regularly update Python runtime, frameworks, and dependencies to supported versions.

Add security testing coverage: Supplement static analysis with dynamic testing, secret scanning, and dependency auditing.

Implement secure logging and error handling: Avoid exposing internal details in exceptions or logs.

Perform periodic manual review: Automated tools may miss logic flaws, insecure configurations,

and contextual risks.

5. Conclusion

The automated scan results do not identify any confirmed codelevel vulnerabilities in the provided Python project. From the perspective of the analyzed source file, the observed security posture is low risk.

That said, the assessment is incomplete because dependency analysis could not be performed due to the absence of a requirements.txt file. This should be addressed promptly to ensure thirdparty component risk is understood and managed. While there is no evidence of immediate critical exposure in the supplied scan output, maintaining dependency visibility is important to prevent overlooked supplychain vulnerabilities.

If you want, I can also convert this into a more formal penteststyle PDFready layout or add an assumptions and limitations section.